

Speedup DP

Longest increasing subsequence

- Given a sequence $A[1..n]$, find the longest subsequence
 - $i_1 < i_2 < \dots < i_m$ such that
 - $A[i_1] < A[i_2] < \dots < A[i_m]$
- $\downarrow$
maximize m

Longest increasing subsequence

- States: $dp[i]$ the longest subsequence that ends at $A[i]$ -- and $A[i]$ is part of the answer
- Decision-making:
 - What will be the second last element in the longest subsequence that ends at $A[i]$
 - Enumerate over all $j < i$ and $A[j] < A[i]$

Longest increasing subsequence

→ longest subsequence end at j

$$\bullet dp[i] = \max_{\substack{j < i \text{ and } A[j] < A[i]}} \{dp[j]\} + 1$$

↓
i will participate

• Time complexity: $O(n^2)$

- n states
- For each state, $O(n)$ choices

↓

$$O(n \cdot \log n)$$

$j \bullet$

$A[i]$

① $j < i$

② $A[j] < A[i]$

Optimization using segment tree

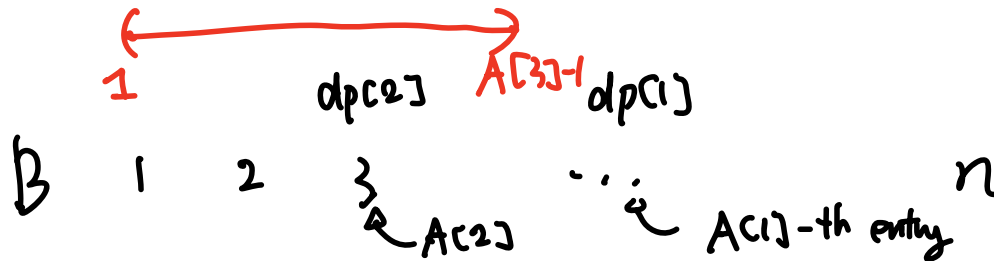
- For now, let us assume every element is between $1 \dots n$
 - We can always discretize every element by sorting $O(n \log n)$

• Recall

$$dp[i] = \max_{j < i \text{ and } A[j] < A[i]} \{dp[j]\} + 1$$

$A[3]$

- Consider another array $B[1..n]$, initialized as $-\infty$



$dp[1]$
 $dp[2]$

Optimization using segment tree

- Every time $dp[j]$ is computed
- * • We will update the value for $B[A[j]] = dp[j]$ if $dp[j] > B[A[j]]$ ①

- To compute $dp[i]$
 - Find the maximum value in $B[1..A[i] - 1]$

Example

At any moment,
 $B[i]$ will record the
largest $dp[j]$
for $A[j] = i$.

maximum value $dp[j]$
for all $A[j] \leq A[i] - 1$
 $= \max_{j < i \text{ and } A[j] < A[i]} \{dp[j]\}$

Optimization using segment tree

- Algorithm:

- Build a segment tree, with every element is $-\infty$
- for $i = 1..n$
 - $dp[i] \leftarrow \text{get_max}(1, A[i] - 1) + 1$
 - $\text{modify}(A[i], dp[i])$
- Return $\max\{dp[i]\}$

} $O(\log n)$

- Time complexity: $O(n \log n)$